

补充模拟卷 03：中间代码表示补缺

目录

A 卷题目	2
B 简明答案	3

A 卷题目

一、四元式、三元式、间接三元式（25 分）

将表达式：

$$x = (a + b) * (c - d) + e$$

翻译为：

1. 普通三地址码。
2. 四元式。
3. 三元式。
4. 间接三元式。

二、函数调用与参数传递（15 分）

将下面语句翻译为三地址码：

$$y = f(a, b + c)$$

要求体现 param 和 call。

三、数组访问与地址计算（20 分）

假设 int 宽度为 4，将下面语句翻译为三地址码：

```
val = a[i]
a[i] = y
```

要求体现下标地址计算。

四、fall-through 与减少冗余跳转（20 分）

为下面语句生成三地址码，尽量利用顺序执行减少冗余 goto：

```
if (x < 100 || x > 200 && x != y)
    x = 0;
```

说明 fall-through 的含义。

五、do-while、break、continue（20 分）

1. 为下面语句生成三地址码：

```
do {  
    a = a + 1;  
} while (a < 10 && b > 0);
```

2. 说明 break 和 continue 在循环中的跳转目标。

B 简明答案

一、四元式、三元式、间接三元式答案

普通三地址码：

```
t1 = a + b  
t2 = c - d  
t3 = t1 * t2  
t4 = t3 + e  
x = t4
```

四元式：

```
(+, a, b, t1)  
(-, c, d, t2)  
(*, t1, t2, t3)  
(+, t3, e, t4)  
(=, t4, -, x)
```

三元式：

```
0: (+, a, b)  
1: (-, c, d)  
2: (*, (0), (1))  
3: (+, (2), e)  
4: (=, (3), x)
```

间接三元式：

三元式表：

```
0: (+, a, b)
1: (-, c, d)
2: (*, (0), (1))
3: (+, (2), e)
4: (=, (3), x)
```

指针表:

```
p0 -> 0
p1 -> 1
p2 -> 2
p3 -> 3
p4 -> 4
```

二、函数调用与参数传递答案

```
t1 = b + c
param a
param t1
t2 = call f, 2
y = t2
```

说明:

先计算实参表达式 $b + c$ 。
再依次传递参数。
`call f, 2` 表示调用 `f`, 参数个数为 2。

三、数组访问与地址计算答案

`val = a[i]:`

```
t1 = i * 4
t2 = a + t1
val = *t2
```

`a[i] = y:`

```
t3 = i * 4
t4 = a + t3
*t4 = y
```

说明:

$i * 4$ 计算偏移量。
 $a +$ 偏移量得到元素地址。
 $*地址$ 表示访问该地址中的值。

四、fall-through 与减少冗余跳转答案

由于 `&&` 优先级高于 `||`，条件按：

```
x < 100 || (x > 200 && x != y)
```

处理。

一种三地址码：

```
if x < 100 goto L_then
if x <= 200 goto L_next
if x == y goto L_next

L_then:
x = 0

L_next:
```

说明：

`x < 100` 为真时直接进入 `then`。
`x < 100` 为假时继续检查 `x > 200 && x != y`。
若 `x <= 200`，则 `x > 200` 为假，直接到 `L_next`。
若 `x == y`，则 `x != y` 为假，直接到 `L_next`。
其余情况顺序落入 `L_then`。

fall-through 的含义：

不显式生成 `goto`，而是让控制流自然顺序执行到下一条指令。

五、do-while、break、continue 答案

三地址码：

```
L_body:
t1 = a + 1
a = t1

if a < 10 goto L_check2
goto L_exit

L_check2:
if b > 0 goto L_body
goto L_exit

L_exit:
```

说明：

`do-while` 是后测试循环，循环体至少执行一次。

`a < 10` 为假时退出。

`a < 10` 为真时才检查 `b > 0`。

`b > 0` 为真时回到循环体，否则退出。

`break` 和 `continue`：

`break` 跳到循环出口。

`continue` 跳到本轮循环结束后的下一次条件检查位置。

对 `while`：

`continue` 通常跳到循环头条件判断处。

`break` 跳到循环后的出口标签。

对 `do-while`：

`continue` 跳到循环底部的条件判断处。

`break` 跳到循环后的出口标签。